
UNIVERSIDADE FEDERAL DO AMAPÁ - UNIFAP
DEPARTAMENTO DE CIÊNCIAS EXATAS E TECNOLOGIAS - DCET
CURSO DE GRADUAÇÃO EM CIÊNCIA DA COMPUTAÇÃO
DISCIPLINA – TEORIA E PARADIGMAS DE LINGUAGENS DE
PROGRAMAÇÃO

Linguagem de Programação Rust

Componentes:

Breno Gabriel de Moraes Duarte | Felipe de Jesus Sodré | Fernando de Jesus Pinto Vasques | Gabriel Canto Góes | Luan Anderson Alves Vieira

Rust é uma linguagem de programação moderna de alto desempenho e focada em segurança de memória, desenvolvida inicialmente pela Mozilla.

Ela é uma linguagem multiparadigma compilada que oferece velocidade comparável a C e C++, mas sem usar um coletor de lixo, conhecido como *garbage collector*.

Utilizando um sistema de propriedade (*ownership*) e checagem de empréstimo (*borrow checking*), o Rust é capaz de gerenciar a memória em tempo de compilação, eliminando erros comuns como ponteiro nulos e estouros de buffer de maneira eficiente.

Legibilidade: Rust possui sintaxe relativamente clara e estruturada, com tipagem explícita, pattern matching, módulos e recursos modernos. Porém, conceitos como *ownership*, *borrowing* e *lifetimes* podem dificultar a leitura para iniciantes.

Redigibilidade: Rust oferece recursos que permitem escrever código expressivo e conciso, como *pattern matching*, *iterators*, *closures* e *type inference*. Entretanto, o sistema de *ownership* exige que o programador escreva código respeitando regras de posse e empréstimo, podendo aumentar a complexidade de certas soluções.

Confiabilidade: O compilador verifica diversas situações que poderiam gerar erros em tempo de execução, especialmente problemas de memória, como *use-after-free*, *double free* e *data races*. O sistema de tipos e o *borrow checker* são elementos centrais dessa característica.

Custo: Rust é gratuito e possui ferramentas oficiais, como Cargo, que facilitam gerenciamento de dependências e compilação. Porém, o aprendizado é mais complexo e a compilação pode ser relativamente custosa, especialmente em projetos grandes.

Estrutura Léxica:

Nome e escopo:

Em Rust, a vinculação (*binding*) entre um nome e uma célula de memória via instrução `let` é **imutável por padrão**. Essa decisão de projeto inverte a tradição de linguagens imperativas como C e Java, priorizando a previsibilidade e a ausência de efeitos colaterais inadvertidos. A mutabilidade de valor deve ser declarada de forma explícita utilizando o modificador `let mut`.

Rust adota o modelo de **escopo estático (léxico)**. No entanto, a linguagem oferece suporte idiomático à **ocultação de nomes (*shadowing*)**: a re-declaração de uma variável utilizando a palavra-chave `let` no mesmo escopo ou em escopos aninhados oculta a declaração anterior. Diferente da alteração de valor em variáveis mutáveis, o *shadowing* aloca um novo identificador e permite alterar o **atributo de tipo** associado ao mesmo nome textual (ex.: converter um `&str` em um `usize`), reduzindo a poluição de nomes temporários na função.

Closures:

Closures são funções anônimas que podem ser armazenadas em variáveis ou passadas como argumentos. A principal diferença para funções comuns é que elas podem capturar valores do ambiente onde foram criadas.

Por padrão, o compilador determina automaticamente como os valores serão capturados. Dependendo de como são utilizados, a captura pode ocorrer por referência compartilhada, referência mutável ou transferência de *ownership*.

O modificador `move` força a closure a capturar os valores por *ownership*:

Tempo de vida e gerenciamento de memória:

Rust não utiliza *garbage collector*. O gerenciamento é baseado em *ownership*, *borrowing* e *lifetimes*, através do sistema de *borrow Checker* do seu compilador.

As regras são: Cada valor possui um proprietário; existe apenas um proprietário por vez; quando o proprietário sai do escopo, o valor é descartado.

Para dados armazenados no *heap*, Rust chama automaticamente *drop* quando o proprietário deixa de existir. Isso permite gerenciamento automático de memória sem um coletor de lixo.

Tempo de vida (*lifetime*) representa o período durante o qual uma referência permanece válida. O compilador verifica esses períodos para impedir referências que apontem para dados que já deixaram de existir.

Sistema de Tipos:

Posição nos dois eixos:

Rust pode ser classificado como

- **Estático:** Pois os tipos são determinados e verificados em compilação.
- **Forte:** Pois não realiza conversões implícitas arbitrárias entre tipos incompatíveis. Quando uma conversão numérica é necessária, normalmente ela deve ser solicitada explicitamente.

Logo, a linguagem Rust se trata de Estático e Forte.

Equivalência de tipos:

Rust utiliza equivalência nominal para seus tipos definidos pelo usuário.

Dois structs com a mesma estrutura não se tornam automaticamente o mesmo tipo apenas porque possuem os mesmos campos. Pessoa e Usuario são tipos diferentes. A estrutura dos campos não é suficiente para torná-los equivalentes.

Isso também é coerente com a organização do sistema de tipos de Rust, que possui tipos definidos pelo usuário como struct, enum e union.

Inferência de tipos:

Rust possui inferência de tipos. O programador não precisa especificar todos os tipos quando o compilador consegue determiná-los. O compilador determina os tipos dessas variáveis a partir dos valores e do contexto.

A inferência também aparece em closures, cujos parâmetros e retorno normalmente não precisam de anotação explícita.

Coerções permitidas:

Rust possui coerções implícitas restritas. Elas só acontecem em situações específicas e entre determinados tipos; não existe conversão automática geral entre tipos incompatíveis.

Entre os exemplos estão:

- $\&\text{mut } T \rightarrow \&T$;
- Coerções envolvendo ponteiros;
- Deref coercion, como $\&\text{String} \rightarrow \&\text{str}$;
- Ou determinadas coerções de tipos dimensionados para não dimensionados (unsizing).

Exemplos:

Demonstração de tipo + inferência:

```
main.rs
1
2 fn main() {
3     let fator = 10;
4
5     let multiplicar = |x| x * fator;
6
7     println!("{}", multiplicar(5));
8
9 }
10
```

```
50
...Program finished with exit code 0
Press ENTER to exit console.
```

Demonstração de escopo de bloco e borrowing. A referência **cópia** deixa de existir ao final do bloco, mas o proprietário **texto** continua válido:

```
main.rs
1
2 fn main() {
3     let texto = String::from("Rust");
4
5     {
6         let copia = &texto;
7         println!("{}", copia);
8     }
9
10    println!("{}", texto);
11
12 }
13
```

```
Rust
Rust
...Program finished with exit code 0
Press ENTER to exit console.
```

Demonstra closure, captura do ambiente e inferência dos tipos:

```
main.rs
1
2 fn main() {
3     let idade: u32 = 20;
4     let nome = "Rust";
5
6     println!("{}", tem {} anos", nome, idade);
7 }
8
```

```
Rust tem 20 anos
...Program finished with exit code 0
Press ENTER to exit console.
```